The Journal of Open Source Software

# k-diagram: Rethinking Forecasting Uncertainty via Polar-based Visualization

**Kouao Laurent Kouadio** [1,2]

**1** School of Geosciences and Info-Physics, Central South University, Changsha, Hunan, 410083, China **2** Hunan Key Laboratory of Nonferrous Resources and Geological Hazards Exploration, Changsha, Hunan, 410083, China

**DOI:** 10.xxxxxx/draft

**Software**

- Review
- Repository
- Archive

**Editor:** Julia Romanowska

**Reviewers:**

- @ChinniAbburi
- @cbueth

**Submitted:** 09 May 2025
**Published:** unpublished

**License**
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

## Summary

`k-diagram` is an open-source Python package designed for the in-depth interpretation and diagnosis of probabilistic forecasts. Moving beyond traditional aggregate metrics, the package provides a suite of novel visualization tools, primarily based on polar coordinate systems, to dissect the complex spatiotemporal structure of forecast uncertainty. By mapping statistical properties such as sharpness, reliability, and temporal stability onto intuitive geometric representations, `k-diagram` enables researchers and practitioners to identify model biases, understand forecast degradation over time, and communicate uncertainty characteristics more effectively. The package is designed to be a practical extension to the standard forecasting workflow (Hong et al., 2025), providing the visual evidence needed for more robust model evaluation and trustworthy, context-aware decision-making.

## Statement of Need

The evaluation of probabilistic forecasts has a rigorous theoretical foundation based on concepts of calibration and sharpness, which are jointly assessed using proper scoring rules (Gneiting et al., 2007). However, these scores are typically aggregated into a single value, which can obscure vital information about a model's performance in heterogeneous spatiotemporal settings. This is a critical limitation, as modern machine learning models, such as Temporal Fusion Transformers (Lim et al., 2021), are increasingly applied to complex, high-dimensional problems where understanding the local and temporal behavior of uncertainty is paramount. A growing body of applied research highlights this challenge in fields as diverse as predictive policing (Rummens & Hardyns, 2021), energy forecasting (Y. Liu et al., 2021), climatology (Baillie & Chung, 2002) and geohazard (J. Liu et al., 2024).

While standard visualization tools exist, they are often not well-suited for the specific task of diagnosing high-dimensional forecast uncertainty. General-purpose libraries like matplotlib (Hunter, 2007) and seaborn (Waskom, 2021) provide the building blocks but lack dedicated functions for these specialized diagnostics. Even established forecasting visualizations like fan charts, which have seen recent innovations (Sokol, 2025), are primarily designed for single time series and do not scale well to problems involving thousands of simultaneous forecasts. This creates a critical gap between the generation of complex probabilistic forecasts and the ability to interpret them effectively.

`k-diagram` addresses this gap by providing a scalable and intuitive toolkit designed specifically for the visual diagnosis of spatiotemporal probabilistic forecasts (J. Liu et al., 2024). The package's novelty lies in its use of polar coordinates to map different dimensions of forecast performance—such as uncertainty magnitude, reliability, and temporal stability—onto angle and radius. This approach provides compact overviews that reveal patterns obscured in traditional

Cartesian plots. By providing clear visual answers to key diagnostic questions (e.g., "Where is a forecast least certain?", "How is its uncertainty evolving?"), `k-diagram` serves as an essential tool for any researcher or practitioner seeking to move beyond aggregate metrics and gain a deeper, more actionable understanding of their forecasting models.

# Functionality

`k-diagram` is implemented in Python (Python Software Foundation, n.d.; Van Rossum & Drake, 2009),, leveraging core scientific libraries including `numpy` (Harris et al., 2020), pandas (McKinney, 2010; Reback et al., 2020), `matplotlib` (Hunter, 2007), `scipy` (Virtanen et al., 2020), and `scikit-learn` (Pedregosa et al., 2011). The core functionality is organized around diagnosing key aspects of forecast quality Figure 1:

- **Uncertainty Diagnostics:** A collection of polar plots under `kdiagram.plot.uncertainty` for visualizing:
  - Prediction interval coverage (point-wise diagnostics and overall scores).
  - Anomaly magnitude (severity and type of interval failures).
  - Interval width (magnitude and consistency over time/samples).
  - Model drift (average uncertainty increase over forecast horizons).
  - Uncertainty drift (evolution of uncertainty patterns using concentric rings).
  - Prediction velocity (rate of change of central forecasts).
- **Model Evaluation:** Functions under `kdiagram.plot.evaluation` for generating standard and enhanced Taylor Diagrams (Taylor, 2001) to compare models based on correlation, standard deviation, and RMSD relative to a reference.
- **Feature Importance:** Radar charts under `kdiagram.plot.feature_based` for visualizing and comparing feature importance profiles ("fingerprints") across different models or contexts.
- **Relationship Visualization:** Polar scatter plots under `kdiagram.plot.relationship` mapping true values to angle and (normalized) predictions to radius.
- **Data Utilities:** Helper functions under `kdiagram.utils` for detecting, validating, and reshaping pandas DataFrames containing quantile-based forecast data, facilitating data preparation for the plotting functions.
- **Command-Line Interface (CLI):** A `k-diagram` command allowing users to generate key plots directly from CSV files without writing Python scripts.
- **Customization:** Plots offer various parameters for adjusting appearance, angular coverage (`acov`), colormaps, normalization, and labeling.

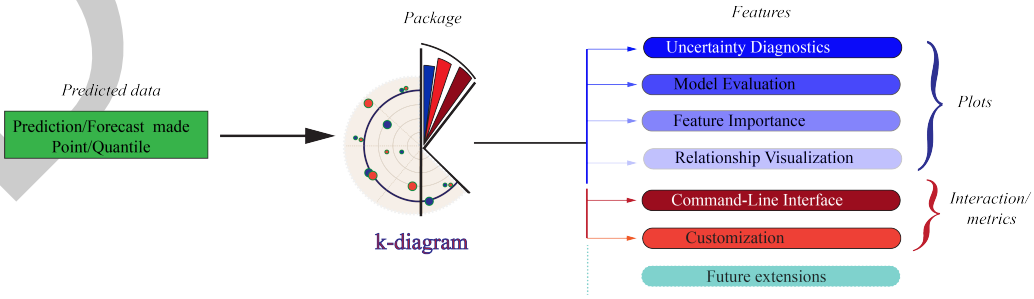


**Figure 1:** Structure of the k-diagram, illustrating the identification of global features.

The package is designed for ease of use and customization, allowing users to control plot aesthetics, angular coverage, and color mapping to tailor the visualizations for their specific domain. Figure @ref(fig:performance) demonstrates some of these key diagnostic plots.

DRAFT

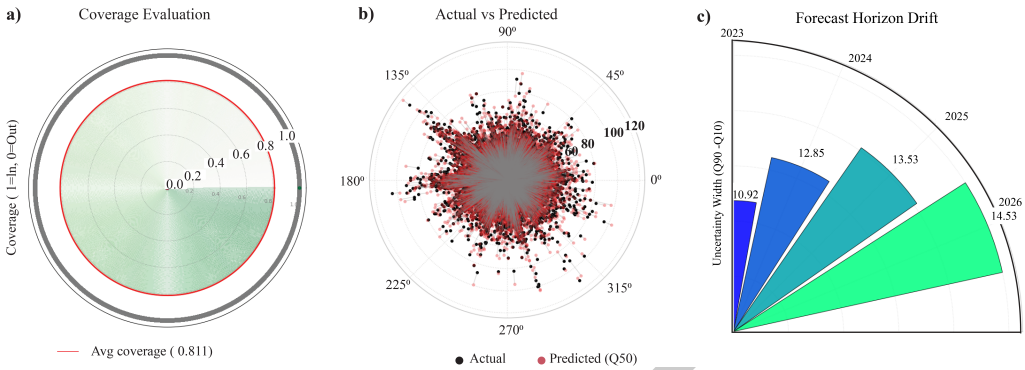


**Figure 2:** Figure 2: Model performance evaluation. (a) Coverage Evaluation: radial plot comparing empirical coverage against nominal quantile levels (average coverage = 0.811). (b) Actual vs Predicted (Q50): polar scatter of observed subsidence (black) and median forecasts (red), with connecting lines. (c) Forecast Horizon Drift: radial bar chart of uncertainty width (Q90–Q10) for forecast years 2023–2026, illustrating increasing prediction uncertainty.

## Installation

The latest stable release of `k-diagram` is available on the Python Package Index (PyPI) and can be installed using pip:

```
pip install k-diagram
```

Detailed installation instructions are available in the documentation.

## Documentation

Comprehensive documentation, including a User Guide explaining the concepts behind the plots, a gallery of examples, and a detailed API reference, is hosted on ReadTheDocs: https://k-diagram.readthedocs.io/.

## License

`k-diagram` is made available under the OSI-approved Apache License 2.0. The full license text can be found in the LICENSE file in the root of the repository.

## Example Application

The visualization methods implemented in k-diagram were developed alongside research applying deep learning models to complex environmental forecasting challenges (L. Kouadio et al., 2025). Specifically, these polar diagnostics were utilized to analyze and interpret the uncertainty associated with land subsidence predictions using an Extreme Temporal Fusion Transformer model (K. L. Kouadio et al., 2025) in Zhongshan city, China.

Full usage examples and a gallery of all plot types are available in the official documentation gallery section.

## Contributing

Contributions to `k-diagram` are highly encouraged! We welcome bug reports, feature requests, documentation improvements, and code contributions. Please refer to the `CONTRIBUTING.rst` guide in the documentation (or repository) for detailed guidelines on how to contribute. Development discussions and issue tracking occur on the GitHub repository.

DRAFT

## Acknowledgements

We extend our sincere gratitude to the anonymous colleagues who provided invaluable feedback during the development of `k-diagram`, as well as those who rigorously tested its early iterations. Their insights and dedication were instrumental in refining the software.

We also appreciate the constructive feedback from early users and reviewers, whose contributions have significantly enhanced the quality and usability of the project.

## References

Baillie, R. T., & Chung, S. K. (2002). Modeling and forecasting from trend-stationary long memory models with applications to climatology. *International Journal of Forecasting*, *18*(2), 215–226. https://doi.org/10.1016/S0169-2070(01)00154-6

Gneiting, T., Balabdaoui, F., & Raftery, A. E. (2007). Probabilistic forecasts, calibration and sharpness. *Journal of the Royal Statistical Society. Series B: Statistical Methodology*, *69*(2), 243–268. https://doi.org/10.1111/j.1467-9868.2007.00587.x

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Memmesheimer, S., Backx, T., & others. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hong, S., Choi, Y., & Jeon, J. J. (2025). Interpretable water level forecaster with spatiotemporal causal attention mechanisms. *International Journal of Forecasting*, *41*(3), 1037–1054. https://doi.org/10.1016/j.ijforecast.2024.10.003

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kouadio, K. L., Liu, Z., Liu, R., Bizimana, P. C., Yang, G., & Liu, W. (2025). *XTFT: A Next-Generation Temporal Fusion Transformer for Uncertainty-Rich Time Series Forecasting*. TechRxiv. https://doi.org/10.22541/au.175390529.91420978/v1

Kouadio, L., Liu, R., Loukou, K. G. H., Liu, J., & Liu, W. (2025). Analytics framework for interpreting spatiotemporal probabilistic forecasts. *International Journal of Forecasting*.

Lim, B., Arík, S. O., Loeff, N., & Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, *37*(4), 1748–1764. https://doi.org/10.1016/j.ijforecast.2021.03.012

Liu, J., Liu, W., Allechy, F. B., Zheng, Z., Liu, R., & Kouadio, K. L. (2024). Machine learning-based techniques for land subsidence simulation in an urban area. *Journal of Environmental Management*, *352*(2024), 17. https://doi.org/10.1016/j.jenvman.2024.120078

Liu, Y., Davanloo Tajbakhsh, S., & Conejo, A. J. (2021). Spatiotemporal wind forecasting by learning a hierarchically sparse inverse covariance matrix using wind directions. *International Journal of Forecasting*, *37*(2), 812–824. https://doi.org/10.1016/j.ijforecast.2020.09.009

McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. 56–61. https://doi.org/10.25080/majora-92bf1922-00a

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Olsczewski, B., Blondel, M., Prejbal, F., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, *12*, 2825–2830.

Python Software Foundation. (n.d.). *Python language reference, version 3.x*. https://dl.acm.org/doi/10.5555/1593511

Reback, J., McKinney, W., Bossche, J. V. den, Augspurger, T., Cloud, P., Sinhrks, & others. (2020). pandas: a foundational Python library for data analysis and manipulation. *Proceedings of the 19th Python in Science Conference*, 1–9. https://doi.org/10.25080/

DRAFT

majora-342d178e-001

Rummens, A., & Hardyns, W. (2021). The effect of spatiotemporal resolution on predictive policing model performance. *International Journal of Forecasting*, *37*(1), 125–133. https://doi.org/10.1016/j.ijforecast.2020.03.006

Sokol, A. (2025). Fan charts 2.0: Flexible forecast distributions with expert judgement. *International Journal of Forecasting*, *41*(3), 1148–1164. https://doi.org/10.1016/j.ijforecast.2024.11.009

Taylor, K. E. (2001). Summarizing quantitative measures of model performance in a single diagram. *Journal of Geophysical Research: Atmospheres*, *106*(D7), 7183–7192. https://doi.org/10.1029/2000JD900719

Van Rossum, G., & Drake, F. L. (2009). *Python 3 reference manual*. CreateSpace. ISBN: 1441412697

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Wehrer, W., Huttunen, J., Håcklin, T., Straub, S., Pike, J., Petroff, A., Conti, K., Lieberman, R., Pollard, J., Nichols, B., Raimi, J., … Vázquez-Baeza, Y. (2020). SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, *6*(60), 3021. https://doi.org/10.21105/joss.03021

DRAFT